

Optimal Resizable Arrays — 统一 Review

论文: Optimal Resizable Arrays (Tarjan & Zwick, 2024) 仓库: `algorithm-pa6-optimal-resizable-arrays`

第1章摘要

动态数组 (Resizable Array) 是一种支持自动调整大小的数据结构。

它具有以下特点:

- 数组长度可以随着元素数量变化动态增加或减少。
- 支持通过索引访问元素。
- 随机访问时间保持为 $O(1)$ 。

Tarjan 和 Zwick 在论文《Optimal Resizable Arrays》中研究了动态数组中的空间与时间权衡问题。

论文的核心思想:

- 将空间划分为存储空间 (storage space) 和临时空间 (temporary space)。
- 利用调整过程中的额外临时空间, 降低长期存储空间需求。

论文证明, 对于任意整数 $r \geq 2$, 可以实现:

项目	复杂度
存储空间	$N + O(N^{1/r})$
临时空间	$N + O(N^{1-1/r})$
随机访问	$O(1)$
扩容/缩容	$O(r)$ 均摊时间

该论文的主要贡献包括：

1. 重新定义动态数组优化问题的研究方向。
2. 在保持高效操作的同时进一步降低存储空间。
3. 给出了动态数组空间和时间复杂度之间更加精细的理论权衡。

从理论角度看，该工作证明了动态数组仍然存在进一步优化空间。

从工程角度看，该论文展示了一种典型设计思想：

通过增加调整过程中的复杂度，换取长期运行时更高的空间利用率。

因此，本文认为《Optimal Resizable Arrays》的价值主要体现在两个方面：

- 推进了动态数组空间优化的理论边界。
- 为其他动态数据结构的空间设计提供了新的优化思路。

第2章引言

动态数组是现代软件系统中的基础数据结构。

例如：

- C++ `vector`。
- Java `ArrayList`。
- Python `list`。

这些实现通常采用类似倍增策略（doubling technique）。

当数组空间不足时：

1. 创建一个更大的连续存储空间。
2. 将已有元素复制到新空间。
3. 释放旧空间。

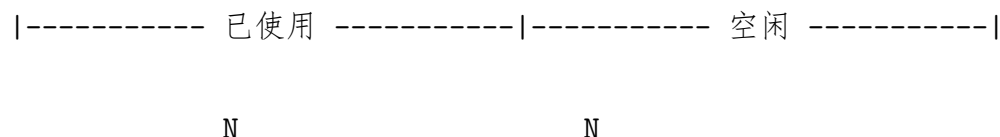
这种方法具有较好的时间效率：

- 单次访问时间为 $O(1)$ 。
- 扩容操作均摊时间为 $O(1)$ 。

但其空间利用率有限：

- 扩容后会保留大量未使用空间。
- 最坏情况下接近一半容量处于空闲状态。

假设当前数组包含 N 个元素，容量可能达到 $2N$ ：



其中一部分空间长期处于未使用状态。

因此，传统动态数组存在一个核心问题：

如何在保持高效访问和调整大小操作的同时，进一步降低存储空间。

Tarjan 和 Zwick 在论文《Optimal Resizable Arrays》中研究了这一问题。

论文的核心思想：

- 将空间划分为存储空间（storage space）和临时空间（temporary space）。
- 利用调整过程中的额外临时空间，降低长期存储空间需求。

论文关注动态数组中的空间和时间权衡。

主要目标：

1. 保持随机访问时间为 $O(1)$ 。
2. 保持 Grow 和 Shrink 操作具有较低均摊时间。
3. 将长期存储空间降低到接近实际元素数量 N 。

Tarjan 和 Zwick 给出的结果如下：

项目	复杂度
Storage Space	$N + O(N^{1/r})$
Temporary Space	$N + O(N^{1-1/r})$
Access	$O(1)$
Grow/Shrink	$O(r)$ 均摊时间

其中参数 r 用于控制空间和时间之间的权衡：

- 增大 r 可以进一步减少存储空间。
- 同时会增加调整过程的时间成本。

该论文的重要性包括：

1. 降低动态数组长期存储空间。
2. 给出空间和时间复杂度之间新的理论界限。
3. 展示通过增加调整成本换取空间优化的方法。

该研究说明，即使是经典数据结构，也存在进一步优化空间。

本文以下章节组织如下：第3章给出预备知识，包括动态数组的形式化定义和空间模型；第4章综述相关工作和研究脉络；第5章分析下界理论；第6章展开上界构造；第7章通过 Growth Game 证明紧致性；第8章进行技术讨论与评价；第9章讨论局限性与开放问题；第10章总结全文。

第3章预备知识

3.1 Resizable Array 定义

Resizable Array 是一种支持动态调整大小的数组结构。

它需要同时满足：

- 数组长度可以动态增加或减少。
- 元素可以通过索引直接访问。
- 访问操作保持较低时间复杂度。

论文研究的动态数组支持以下基本操作：

操作	功能
Array()	创建空数组
Length()	返回当前数组长度
Get(i)	返回索引 i 位置元素
Set(i,x)	修改索引 i 位置元素
Grow(x)	在数组末尾增加元素
Shrink()	删除数组末尾元素

动态数组的核心目标是在保持访问效率的同时，降低扩容和缩容产生的额外空间开销。

3.2 空间模型

论文将动态数组占用的空间分为两类：

Storage Space

Storage Space 指数组稳定运行状态下保存元素所需要的空间。

对于大小为 N 的数组：

$$N + s(N)$$

其中：

- N 表示实际存储元素数量。
- $s(N)$ 表示额外存储空间。

Temporary Space

Temporary Space 指扩容或缩容过程中短暂使用的额外空间。

对于大小为 N 的数组：

$$N + t(N)$$

该模型允许调整过程中使用更多临时空间，从而降低长期存储空间需求。

第4章相关工作

动态数组的发展经历了从简单倍增方案到空间优化结构的发展过程。

整体发展路线：

Doubling

↓

Sitariski HAT

↓

Brodnik 等人的空间下界研究

↓

Tarjan-Zwick Optimal Resizable Arrays

4.1 传统倍增方法 (Doubling)

传统动态数组通常维护一个连续存储空间。

当容量不足时：

- 创建更大的数组。
- 复制已有元素。
- 释放旧数组。

特点：

项目	表现
访问时间	$O(1)$
扩容时间	$O(1)$ 均摊
实现复杂度	低

优势：

- 实现简单。
- 工程应用广泛。

限制：

- 存储空间可能达到 $2N$ 。
- 存在较大的空闲空间。

4.2 Sitarski 的 HAT Data Structure

Sitarski 提出了 HAT (Hierarchical Array Tree) 结构。

核心思想：

- 将数组拆分成多个存储块。
- 使用额外索引结构管理这些块。

特点：

项目	表现
存储空间	$N + O(\sqrt{N})$
操作时间	$O(1)$

相比传统倍增方法，HAT 降低了空间浪费。

4.3 Brodник 等人的研究

Brodnik 等人进一步研究动态数组的空间下界问题。

他们证明：

在限制额外临时空间的情况下，支持动态调整大小和随机访问的数据结构需要至少：

$$N + \Omega(\sqrt{N})$$

的额外空间。

这一结果说明：

- HAT 类结构已经达到渐近最优。
- 进一步降低存储空间需要新的空间模型。

4.4 Tarjan 和 Zwick 的改进

Tarjan 和 Zwick 重新设计了空间模型。

他们将空间分为：

- Storage Space：长期保存数组元素的空间。
- Temporary Space：调整过程中使用的临时空间。

通过允许更多临时空间，论文突破了传统空间限制。

其结果：

项目	复杂度
Storage Space	$N + O(N^{1/r})$
Temporary Space	$N + O(N^{1-1/r})$
Access	$O(1)$
Grow/Shrink	$O(r)$ 均摊

参数 r 控制两者之间的平衡：

- r 较大时，Storage Space 更接近 N 。
- 同时调整操作成本增加。

4.4.1 论文发表后的后续进展

Tarjan 和 Zwick 的研究进一步推动了动态数组空间优化方向的发展。

该论文最初以预印本形式公开，随后发表于 SIAM Journal on Computing。

论文正式发表后，其提出的 Storage Space 与 Temporary Space 分离思想成为分析动态数组空间效率的重要方法。

(1) 工程实现方向 传统动态数组主要采用倍增策略。

典型实现包括：

- C++ vector。
- Java ArrayList。
- Python list。

这些结构具有以下特点：

- 实现简单。
- 访问效率高。
- CPU 缓存局部性较好。

Optimal Resizable Arrays 通过更复杂的数据结构降低空间浪费。

这种方法需要维护：

- 多级数据块。

- 额外索引结构。
- 动态调整过程。

因此，在实际工程应用中，需要进一步考虑：

- 内存分配成本。
- 数据访问模式。
- CPU 缓存效率。
- 实现维护成本。

(2) 理论推广方向 论文提出的核心思想是：

通过增加调整过程中的临时空间，降低长期存储空间需求。

这一思想为其他空间高效数据结构提供了新的设计方向。

相关研究方向包括：

- 紧凑型动态容器。
- 外部存储数据结构。
- 内存受限环境中的数据管理。

(3) 当前影响 目前，Optimal Resizable Arrays 主要影响体现在两个方面：

1. 理论分析方面。

该工作提供了新的动态数组空间模型，重新分析了存储空间和调整空间之间的关系。

2. 数据结构设计方面。

该工作展示了经典数据结构仍然存在进一步优化空间。虽然其复杂实现限制了大规模工程应用，但它为未来空间高效数据结构设计提供了理论基础。

4.5 相关工作总结

方法	存储空间	调整时间	特点
Doubling	$O(N)$	$O(1)$ 均摊	简单，工程中常用

方法	存储空间	调整时间	特点
Sitarski HAT	$N + O(\sqrt{N})$	$O(1)$	降低空间浪费
Brodnik 等结构	$N + O(\sqrt{N})$	$O(1)$	证明空间下界
Tarjan-Zwick	$N + O(N^{1/r})$	$O(r)$	利用临时空间突破限制

Tarjan 和 Zwick 的工作建立在此前动态数组研究基础上，通过重新划分空间模型进一步推进了动态数组优化的理论边界。

第5章下界理论

5.1 抽屉原理证明 Brodnik 下界

Brodnik等人（1999）证明了：任何可伸缩数组实现，都必须在某些时刻使用 $N + \Omega(\sqrt{N})$ 空间

定理 4.1 (Brodnik et al.): 任何支持 access 与 grow 操作的可伸缩数组，存在某个时刻，已分配的总空间至少为 $N + \Omega(\sqrt{N})$

证明思路（抽屉原理）：

考虑连续执行 N 次 grow 操作后的状态。元素被存储在 k 个内存块中。分两种情况论证：

- $k \geq \sqrt{N}$ ：每个块都需要一个指针指向该块的起始地址。仅指针数组本身就占用了至少 $k \geq \sqrt{N}$ 的空间，而这些指针构成纯粹额外开销
- $k < \sqrt{N}$ ：由抽屉原理， N 个元素分入少于 $\sqrt{N}$ 个块中，必存在某个块的大小超过 $N/k > \sqrt{N}$ 。该块刚被分配的时刻——即该块为空时，该空块本身构成超过 $\sqrt{N}$ 的临时额外空间

两种情况均导致至少 $\Omega(\sqrt{N})$ 的额外空间，该定理得证

这一定理为后续所有工作设置了基准线——在不区分”常驻空间”与”临时空间”的前提下， $O(\sqrt{N})$ 的额外空间是绕不开的硬下界

Tarjan 与 Zwick 将上述经典下界推广到常驻额外空间 $s(N)$ 与临时额外空间 $t(N)$ 之间的乘积下界

定理 4.2: 任何 $(s(N), t(N))$ -implementation 必须满足 $s(N) \cdot t(N) \geq N$ (忽略常数因子)

完整:

1. 连续执行 N 次 grow 操作。此时 N 个元素被存放在某个集合的内存块中，设块的数量为 k
2. 每个已分配的块需要一个指针来记录其地址。这些指针存储在索引结构中，占用至少 k 个 word。因此指针开销本身已构成额外空间的下界： $k \leq s(N)$
3. 由抽屉原理， N 个元素分布在 k 个块中，必存在某个块的大小至少为 $\lceil N/k \rceil \geq N/s(N)$
4. 考虑该最大块的分配时刻。在刚分配完成未填入任何元素时，该块的全部 $N/s(N)$ 个 word 均为“临时额外占用”的空间
5. 由定义，grow 过程中的临时额外空间不得超过 $t(N)$ ，因此 $t(N) \geq N/s(N)$
6. 移项即得 $s(N) \cdot t(N) \geq N$ 。推导完毕

推论 4.3: 若 $s(N) = O(N^{1/r})$ ，则必有 $t(N) = \Omega(N^{1-1/r})$

推导: 很简单，将 $s(N) = O(N^{1/r})$ 代入定理 4.2 即可：

$$t(N) \geq \frac{N}{s(N)} \geq \frac{N}{c \cdot N^{1/r}} = \frac{1}{c} \cdot N^{1-1/r} = \Omega(N^{1-1/r})$$

下面几章证明这个下界是紧的。

第6章上界构造

6.1 $r=3$ 特例构造

论文先用 $r=3$ 的特例来帮助理解

6.1.1 数据结构与空间分析

设参数 B 满足 $N^{1/3} \leq B \leq 4N^{1/3}$ ，于是有：

- **Large blocks:** 大小为 B^2 ，稳态下总是满的，数量至多 B 个
- **Small blocks:** 大小为 B ，最多 $2B$ 个，仅最末一块可部分填充。充当“缓冲区”吸收局部变化，避免频繁触及 large block 层
- **索引指针:** 合计 $O(B) = O(N^{1/3})$ 个
- **稳态额外空间:** $s(N) = O(N^{1/3})$ （指针 + 至多一个未满 small block）
- **临时额外空间:** $t(N) = O(B^2) = O(N^{2/3})$ （合并/拆分时分配的新块）

这个数据结构类似于一个类似 2 位的冗余 B 进制计数器

6.1.2 核心操作

Access（访问元素）:

元素在块内外的顺序与逻辑顺序一致，该数据结构寻址等同于二级索引，所以时间复杂度是为 $O(1)$

Grow（追加元素）:

1. 最末 small block 未满 → 直接写入， $O(1)$
2. Small block 已满但数量未达 $2B$ → 分配新 small block 写入， $O(1)$
3. Small blocks 满 $2B$ 个 → 触发 **Combine**: 取 B 个满载 small block，将其全部元素拷贝到一个新分配的 B^2 large block 中，释放旧 small block，新 large block 追加到高层，类似于 B 进制计数器的**进位**——低位积累满 B 个单元时合并为一个高位单元

Shrink（删除末尾元素）:

1. 最末 small block 非空 → 直接删除， $O(1)$ 。
2. 最末 small block 为空 → 触发 **Split**: 取最末 large block，将其末尾 B 个元素拆分到 B 个新 small block 中，释放原 large block，然后从 small block 层删除，类似于 B 进制计数器的**借位**——将一个大额面值拆为小面值

6.1.3 时间复杂度

操作	复杂度	说明
access	$O(1)$	两级索引，常数时间定位
grow/shrink (普通)	$O(1)$	直接在小块层操作

6.2 一般 r 级构造

将 $r=3$ 推广到任意整数 $r \geq 2$, $r=3$ 模拟的是一个** 2 位的冗余 B 进制计数器，推广到 $r \geq 2$ 模拟的是一个 $r-1$ 位的冗余 B 进制计数器**

6.2.1 数据结构与空间分析

设参数 B 满足 $N^{1/r} \leq B < 4N^{1/r}$ (为 2 的幂), 第 i 层 ($i = 1, \dots, r-1$) 由大小为 B^i 的数据块组成。与§6.1 ($r=3$) 的对比如下:

	§6.1 ($r=3$ 特例)	§6.2 (一般 r)	说明
层数	2 层 (small / large)	$r-1$ 层	-
块大小	B, B^2	$B, B^2, \dots, B^{r-1}$	B^i 幂次序列
每层容量	各至多 $2B$ 个块	各至多 $2B$ 个块	不变, 始终是冗余 B 进制
满载层	large block (第 2 层)	第 2 至 $r-1$ 层	最高层满载 → 除第 1 层外均满载
缓冲区	small block 层 (第 1 层)	第 1 层 (块大小 B)	始终是底层的最小块进行增删的缓冲
索引指针	$O(B) = O(N^{1/3})$	$O(rB) = O(rN^{1/r})$	指针数随层数线性增长
$s(N)$	$O(N^{1/3})$	$O(rN^{1/r})$	$N^{1/3} \rightarrow N^{1/r}$, 多因子 r
$t(N)$	$O(N^{2/3})$	$O(N^{1-1/r})$	$N^{2/3} \rightarrow N^{1-1/r}$
$s(N) \cdot t(N)$	$O(N)$	$O(rN)$	均达到定理 4.2 的 $\Theta(N)$ 下界

6.2.2 核心操作

Access（访问元素）：

元素在块内外的顺序与逻辑顺序一致，通过位运算（msb、移位等）在 $O(1)$ 时间内定位下标 i 所属的层和块内偏移。

Grow（追加元素）：

1. 第 1 层有空位 $\rightarrow$ 直接写入最末块， $O(1)$
2. 第 1 层满但未达 $2B \rightarrow$ 分配新 B 大小块写入， $O(1)$
3. 第 1 层满 $2B$ 块 $\rightarrow$ 触发 **Combine**：找首个有空位的高层 k ($n_k < 2B$)，对 $i = k - 1$ 到 1，将第 i 层 B 个满块合并为一个 B^{i+1} 大小的块，释放旧块，新块提升到第 $i + 1$ 层。类似于 B 进制计数器的**进位**——低位积累满 B 个单元时合并为一个高位单元

Shrink（删除末尾元素）：

1. 第 1 层有元素 $\rightarrow$ 直接删除末尾元素， $O(1)$
2. 第 1 层无块 $\rightarrow$ 触发 **Split**：找最底层有块的高层 k ($n_k > 0$)，将其一个 B^k 块拆为 B 个 B^{k-1} 块，逐级下传至第 1 层，然后删除。类似于 B 进制计数器的**借位**——将一个大额面值拆为小面值

Rebuild（全局重建）：

当 N 超出 $[B^r/8, B^r]$ 时， B 加倍或减半，逐块搬运重建。每次仅分配一个 B^{r-1} 新块，搬空旧块立即释放，确保重建中空间始终不超过 $N + O(N^{1-1/r})$ 。

6.2.3 均摊分析（Credit 论证）

核心思想：考虑未来可能发生的拷贝代价提前分配 credit

Grow 方向：每个新元素分配 $r - 1$ 个 credit 进入第 1 层。每上升一层消耗 1 个 credit 支付拷贝代价。credit 始终非负——第 1 层最多 $(r - 1)$ ，第 $r - 1$ 层最少 (1) **不变量**：元素 credit 数 = $r -$ 所在层数。均摊代价 = $O(1) + r - 1 = O(r)$

Shrink 方向：每次 shrink 向每层分配 $O(1)$ credit。层 k 的 Split 拷贝代价 $O(B^k)$ 由该层积累的 credit 支付；两次同层 Split 之间至少经历 B^k 次 shrink，足够涵盖拷贝代价

Rebuild：代价 $O(N)$ ，间隔 $\Omega(N)$ 次操作，摊还 $O(1)$

6.2.4 时间复杂度

操作	复杂度	说明
access	$O(1)$	位运算定位，常数条机器指令
grow/shrink（普通）	$O(1)$	直接在第 1 层操作
Combine/Split	均摊 $O(1)$	进位/借位代价由 credit 覆盖
Rebuild	均摊 $O(1)$	代价 $O(N)$ ，每 $\Omega(N)$ 次操作触发

6.3 空间优化变换

§6.2 给出 $s(N) = O(rN^{1/r})$ 带了一个因子 r 。当 r 为固定常数时无所谓，仍为最优实现，但如果 r 随着 N 增长（例如 $r = \log \log N$ ），这个 r 就不能简单地直接忽略。于是本节证明额外空间量可以降至 $s(N) = O(N^{1/r})$

6.3.1 变换针对什么时机？

§6.2 的数据结构是标准的：每次 Combine 会分配一个 B^{i+1} 大小的新块，把 B 个 B^i 小块中的元素挨个拷贝进去，然后释放那些小块。在“新块刚分配好、旧块还没释放”的瞬间，临时额外空间达到了峰值——这就是 $t(N) = O(N^{1-1/r})$ 。

变换的时机就是在这个分配的瞬间。取 $T = N^{1-1/r}$ （即临时空间峰值的量级）。每当结构要向系统申请一个大小为 b 的新块时：

- 如果 $b \leq T$ ，正常分配，一个块搞定。
- 如果 $b > T$ ，不分配一块大的，改为分配 $\lceil b/T \rceil$ 个大小为 T 的块——相当于把那个大块切成了一串小方块。元素照旧按顺序填进去，读完一块接着下一块。

功能完全等价：拷贝总量不变，元素顺序不变，access 多一步块间跳转但仍然是 $O(1)$ 。

6.3.2 为什么指针反而变少了？

关键后果：变换之后，系统里没有任何块的尺寸超过 T 。所有已分配空间——数据块、索引块、元数据——每个块的大小 $\leq T$ 。

原来§6.2 的指针开销是分层的： $r - 1$ 层索引，每层各自维护至多 $2B$ 个指向数据块的指针，层级叠加出 r 因子。变换之后分层没意义了——所有块都是 T 大小，一层统一管理即可。总块数（也就是总指针数）不超过总分配空间除以 T ：

$$\text{总块数} \leq \frac{N + s(N) + t(N)}{T} \approx \frac{N}{N^{1-1/r}} = N^{1/r}$$

6.3.3 形式化

论文将上述操作归纳为一个通用引理，然后将它作用到§6.2 构造的一个变体上：

引理 7.2： 设有标准 $(s(N), O(N))$ -实现。对任意 $t(N)$ ，可将其所有“分配新块”操作中的大块替换为 $t(N)$ 大小的子块，得到 $s'(N) = s(N) + O(N/t(N))$ 、 $t'(N) = s(N) + O(t(N))$ 的新实现，时间界不变。

定理 7.3： 取§6.2 中参数为 $2r$ 的构造 ($s = O(rN^{1/(2r)}), t = O(N^{1-1/(2r)})$)，对其施加引理 7.2（取 $t(N) = N^{1-1/r}$ ），得 $s'(N) = O(rN^{1/(2r)} + N^{1/r})$ 。在 $r \leq \frac{1}{2} \frac{\log N}{\log \log N}$ 条件下， $rN^{1/(2r)} \leq N^{1/r}$ ，故 $s'(N) = O(N^{1/r})$ 、 $t'(N) = O(N^{1-1/r})$ 。

6.3.4 实践意义

r 随 N 增长的场景：例如 $r = \Theta(\log N)$ 时 $rN^{1/r} = \Theta(\log N)$ ，变换后降到 $N^{1/r} = \Theta(1)$ ——把额外空间压到了常数级。

第7章紧致性证明

第6章证明了 $O(r)$ 均摊的上界。本章使用一个抽象博弈证明无法更好了——其推论直接给出 $\Omega(r)$ 的下界。

7.1 Growth Game 模型与精确解

7.1.1 博弈定义

博弈设定： N 个元素逐个插入，玩家维护 k 个子数组 $A_1, \dots, A_k$ （初始为空），最多允许 ℓ 个空位。空子数组总是在最前面，第一个非空子数组至多 ℓ 个空位，其余全满。插入时分三种情况：

情况	操作	代价
(a) 有空位	直接写入	1
(b) 有空子数组	分配大小 $\ell + 1$ 的新子数组，写入	1
(c) 全部 k 个满	选 $i \in [k]$ ，合并 $A_i, \dots, A_1$ 到新块，写入	$1 + \sum_{j=1}^i a_j$

(a)(b) 无选择。博弈的核心在 (c)： k 种合法移动——选 $i = 1$ 代价小但碎片多，选 $i = k$ 代价大但干净。目标：使总代价 $C_{N,k,\ell}$ 最小。

7.1.2 博弈分析

第一步：消除 ℓ 。 每次分配新子数组留下 ℓ 个空位后，接下来 ℓ 次插入必然直接填空，不需要决策。每 $\ell + 1$ 次操作中仅 1 次是真正的决策。将 $\ell + 1$ 个元素打包为一个逻辑单元，博弈退化为无空位版本：

$$C_{N,k,\ell} = (\ell + 1) \cdot C_{N/(\ell+1),k,0}, \quad A_{N,k,\ell} = A_{N/(\ell+1),k,0}$$

此后只讨论 $\ell = 0$ ，记 $C_{N,k} = C_{N,k,0}$ 。

第二步：建立递推。 状态用向量 $\mathbf{a} = (a_1, \dots, a_k)$ 描述（ $\sum a_i = N$ ，空子数组在前缀）。令 $C(\mathbf{a})$ 为从全零到达 $\mathbf{a}$ 的最小代价。到达 $\mathbf{a}$ 的最后一步必定是最后一次改变 A_k 的合并，它将系统带入 $(0, \dots, 0, a_k)$ 态，此后 A_k 不再变化。由此（引理 8.4）：

$$C_k(a_1, \dots, a_k) = C_k(0, \dots, 0, a_k) + C_{k-1}(a_1, \dots, a_{k-1}) \quad (1)$$

$$C_k(0, \dots, 0, a_k) = C_{a_k-1,k} + a_k \quad (2)$$

(1) 将代价拆为“到达 a_k “+”前 $k-1$ 位独立操作”两部分。(2) 刻画最后一步：合并前 a_k-1 个元素所在的全部子数组，代价 a_k ，合并前的代价为 $C_{a_k-1,k}$ 。反复展开得到独立求和式，将任意状态的代价关联到更小参数的最优代价——归纳证明的骨架。

第三步：闭式解。 通过对 (N, k) 双层归纳求解递推。令 n 满足 $\binom{n+k-1}{k} - 1 \leq N \leq \binom{n+k}{k} - 1$ ，定理 8.6 给出：

$$C_{N,k} = (N+1)n - \binom{n+k}{k+1}, \quad A_{N,k} = \frac{C_{N,k}}{N} \geq \frac{n-1}{2}$$

最优状态下子数组大小被二项式系数夹逼： $\binom{n+i-2}{i} \leq a_i \leq \binom{n+i-1}{i}$ ——“大块多大、小块多小”由组合数决定，不是随意定的。证明核心是扰动论证：若 a_i 偏离上述范围，挪一个元素即可严格降低总代价。

第四步：构造策略。 最优代价可以达到的。策略维护二项式计数器 $(b_1, \dots, b_k)$ ，子数组大小 $a_i = \binom{b_i+i-1}{i}$ 。每次找最小的 i 使 $b_i < b_{i+1}$ ，合并 $A_1, \dots, A_i$ ， b_i 加一、低位归零。这可以看作 §6.2 中 B 进制计数器的理论最优版——进位基数不固定，由二项式系数动态决定（§6.2 的 $2B$ 冗余是为同时支持 grow/shrink 的妥协；这里只有 grow，可达精确最优）。

7.2 归约至数据结构 $\Omega(r)$ 下界

归约分四步：

第一步：块 $\rightarrow$ 子数组。 数据结构中维护的每个内存块对应博弈中的一个子数组 A_i ，块中元素数即 a_i 。当数据结构触发 Combine 合并若干块时，对应博弈中的 (c) 类合并操作，元素拷贝量恰好等于博弈中的移动代价。

第二步：额外空间 $\rightarrow k$ 和 ℓ 。 常驻额外空间 $s(N) = O(N^{1/r})$ 意味着块的数目受限于 $O(N^{1/r})$ 。更精细地分析，多层索引结构使总块数 $k \approx r \cdot N^{1/r}$ 。部分填充的块对应空位 ℓ ，二者量级相当。

第三步：映射到博弈下界。 N 次 grow 操作自然诱导一个 (N, k, ℓ) -增长博弈的合法策略。由归约 $\ell \rightarrow 0$ 及推论 8.13，该博弈的均摊代价满足 $A_{N,k} \geq (n-1)/2$ 。当 $k = \Theta(rN^{1/r})$ 时，由二项式系数的渐近性质可得 $n = \Theta(r)$ ，故 $A_{N,k} = \Omega(r)$ 。

第四步：结论。 博弈的均摊代价是数据结构真实拷贝次数的下界，因此：

定理 9.1: 当 $r = O(\log N)$ 时, 任何使用 $N + O(rN^{1/r})$ 存储空间的标准实现, 其 grow 操作的均摊代价为 $\Omega(r)$ 。

完整归约链条为:

标准数据结构 ($N + O(rN^{1/r})$ 存储)
 $\downarrow$ 块 $\rightarrow$ 子数组, 合并操作 $\rightarrow$ 博弈移动
 $\downarrow$
 (N, k, ℓ) -增长博弈 ($k \approx rN^{1/r}$, $\ell \approx N^{1/r}$)
 $\downarrow$ 引理 8.2: 吸收空位
 $\downarrow$
 $(N/(\ell + 1), k)$ -增长博弈 ($\ell = 0$)
 $\downarrow$ 定理 8.6 + 推论 8.13: $A_{N,k} = \Omega(r)$
 $\downarrow$
 Grow 均摊代价 = $\Omega(r)$

结合第 6 章的上界 $O(r)$, 论文在 grow/shrink 的均摊时间上达到了紧致 (tight)。下表汇总全部维度的上下界对比:

维度	上界 (第 6 章)	下界	结论
稳态存储	$N + O(N^{1/r})$	$\geq N$ (显然)	渐近最优
临时 resize 空间	$N + O(N^{1-1/r})$	$N + \Omega(N^{1-1/r})$ (定理 4.2)	紧致
Grow/Shrink 时间	$O(r)$ 均摊	$\Omega(r)$ 均摊 (定理 9.1)	紧致
Access 时间	$O(1)$ 最坏	$\Omega(1)$ (显然)	最优

论文在存储空间、临时空间、更新时间三个维度上同时达到渐近最优——使”Optimal Resizable Arrays”名副其实

第8章讨论与评价

通读论文其中最重要的几个关键设计:

(1) 把存储空间和临时空间分开。Brodnik 的下界说”不管什么时候, 额外空间至少 $\Omega(\sqrt{N})$ ”。但实际场景中, 一个程序有上百个数组, 只有正在 resize 的那个才需

要临时空间，其余都不需要。把两者分开考虑就可以绕开了这个下界——定理 4.2 的 $s(N) \cdot t(N) \geq N$ ：二者乘积受限，但各自有可调节余量

(2) 用 B 进制计数器来组织块。两级结构 (B 和 B^2) 能把额外空间压到 $O(\sqrt{N})$ 。多叠几层—— $B, B^2, \dots, B^{r-1}$ 便可把空间继续下压，每 B 个小块合并成一个更大的块，与进位逻辑相同。层数越多空间越省，但每个元素最多被搬 $r - 1$ 次——得到了 $O(r)$

(3) 每层允许 $2B$ 个块而不只是 $B - 1$ 。 $B - 1$ 是标准 B 进制计数器的容量，但那样只能 grow 不能 shrink。多留一倍的缓冲空间，Shrink 拆下来的小块有地方放，不会刚合并完又拆开。所以使用冗余 B 进制计数器进行双向缓冲

(4) 用 credit 进行均摊分析。Grow 的 credit 在元素层面——每升一层花 1 个 credit，不变量自己就是证明。Shrink 的 credit 在层的层面——每层靠 shrink 的“存款”来支付 Split。Rebuild 再额外存一份。三方的 credit 各自独立，加起来得到了 $O(r)$ 。

第9章局限性与开放问题

9.1 下界的适用范围

论文证明了 grow 操作的均摊时间下界为 $\Omega(r)$ ，但该下界只适用于论文定义的 standard implementation。

也就是说，论文尚未完全排除非标准实现获得更好结果的可能。作者认为，若对元素和指针加入合适的不可压缩性假设，非标准实现也不会有渐近优势。

因此，一个开放问题是：如何把 $\Omega(r)$ 的下界推广到更一般的算法模型。

9.2 Growth Game 证明较复杂

为了证明下界，论文构造并精确求解了 Growth Game。该博弈描述了连续插入时如何合并数据块，才能使总复制成本最小。

精确解需要较复杂的组合分析。论文指出，若只需要渐近意义上的 $\Omega(r)$ 下界，可能存在更简单的归纳或势能证明方法。

9.3 工程实现的限制

该结构需要维护多级数据块、索引结构、Combine、Split 和重建过程，因此实现比传统倍增数组复杂。

参数 r 也需要在空间和时间之间取舍： r 较大时，常驻存储空间更接近 N ，但 grow 和 shrink 的均摊时间会增加到 $O(r)$ 。未来可进一步比较该结构与传统倍增数组、HAT 等结构在真实内存分配器和缓存环境下的表现。

第10章总结

Tarjan 和 Zwick 研究了动态数组的空间与时间权衡问题。传统倍增数组访问快、扩容均摊时间低，但会保留较多空闲空间。

论文的核心改进是区分 Storage Space 和 Temporary Space：允许扩容或缩容时使用更多临时空间，从而减少稳定状态下的长期存储空间。

论文证明，对于参数 $r \geq 2$ ，可以实现：

常驻存储空间 $N + O(N^{1/r})$ ，临时空间 $N + O(N^{1-1/r})$ ，grow/shrink 均摊时间 $O(r)$ 。

同时，随机访问保持为 $O(1)$ 最坏时间。Growth Game 又给出 grow 操作的 $\Omega(r)$ 均摊时间下界，因此该结果在论文的标准模型中达到了紧致的空间—时间权衡。

该工作说明：经典数据结构仍然可以通过重新划分资源模型获得新的优化结果。虽然结构实现较复杂，但它为未来空间高效数据结构的设计提供了理论基础。